Course Project

Design & Implementation of a Mini Programming Language

A language-design project covering Chapters 1–7 of Sebesta's Concepts of Programming Languages (12th ed.)

Team size	Solo
Single submission deadline	**22 May 2026, 23:59**
Oral defense	Week of 1–5 June 2026 (scheduled slots)
Deliverables	5 artifacts + oral defense (see §3)
AI tool use	**Mandatory**, must be documented

1. Project Overview

In this project you will **design your own small programming language** (a domain-specific language, or DSL) and implement an interpreter for it. The project is intentionally structured so that no AI tool can complete it end-to-end: the core deliverable is **your design decisions** and your ability to **defend them** using the vocabulary of Sebesta's textbook.

You are **required** to use AI assistants (Claude, ChatGPT, Copilot, Gemini, etc.) during the project, and you must document that use in an AI Usage Journal. The goal is not to prevent AI-assisted work but to train you to use AI *as an engineer does* — critically, with judgment, and with full understanding of the output.

> **The one rule that makes this project AI-resistant**
> All deliverables are submitted together on **22 May 2026**, but each one is graded separately and each is designed to be hard to produce convincingly without having done the earlier ones first. Most importantly, every submission is followed by a **15-minute oral viva** in the week of 1–5 June 2026. You must be able to answer questions of the form: *"Why did your language make this choice instead of that one? What would break if you did it the other way?"* If you cannot defend a design decision in person using Sebesta's terminology, that portion of your grade will reflect it — regardless of how polished your submission looks on paper. The oral defense carries substantial weight in the final grade.

2. Choosing Your Language Domain

Each student/pair must choose a **distinct, non-trivial application domain**. No two teams may submit the same domain; domain selection is on a first-come basis and must be approved by the instructor by **10 April 2026**.

Acceptable examples

- **TurtleLang** — a language for drawing geometric shapes with turtle graphics, with recursion and user-defined shape procedures.

- **ChessDSL** — a language for describing and solving chess puzzles (piece placement, legal moves, constraint expressions).
- **QueryLite** — a mini-query language over CSV files, with filters, joins, and aggregates.
- **MusicScript** — a language for describing melodies (notes, durations, loops, transposition).
- **CircuitLang** — a language for describing digital circuits (gates, wires, simulation).
- **RecipeLang** — a language for scaling and substituting ingredients in recipes, with unit types.
- **PlotDSL** — a language for declarative 2D data plots with statistical transforms.

Minimum requirements for any chosen domain

- At least **three primitive types** and **one structured type** (array, record, or user-defined).
- At least **one control structure** beyond sequencing (if, while, for, or pattern-match).
- **User-defined functions or procedures** with parameters.
- At least **one domain-specific construct** that justifies why this is a *DSL* and not a generic scripting language.
- A nontrivial expression language with operators that raise interesting design questions (precedence, associativity, mixed-mode arithmetic, coercion).

3. Deliverables

All five written artifacts are submitted together as a single zip archive on Monday, 25 May 2026 by 23:59. The oral defense takes place in the week following the deadline. Each artifact is graded separately.

#	Artifact	Format
D1	**Design Specification** Domain, EBNF grammar, type system, scoping, semantics for at least two constructs, and per-decision rationale tied to Sebesta Ch. 1–7.	PDF, 8–12 pages
D2	**Implementation** Source code: lexer, parser, type checker, interpreter. Runs the three example programs of D3 end-to-end from the command line.	Git repo
D3	**Example Programs & Test Report** Three non-trivial programs in your language with expected outputs, plus five malformed programs showing parser error recovery.	Source + PDF, 3–5 pages
D4	**AI Usage Journal** Documented record of your AI-tool use across the project, including the three mandatory experiments of §6.	PDF, 6–15 pages
D5	**Retrospective & Self-Assessment** One-page honest retrospective: what you would change, what you could not get working, and a self-grade against the rubric in §5 with justification.	PDF, 1 page
OD	**Oral Defense** 15-minute viva in the week of 26–30 May: 5 minutes live demo + 10 minutes of questions about your design decisions and implementation. Schedule will be posted by 20 May.	In person

Late policy as described by the course syllabus.

Why the single deadline does not make this an "all-nighter project"

The five artifacts depend on each other in ways that cannot be faked at the last minute. Your **Implementation** (D2) must be consistent with your **Design Specification** (D1) — contradictions are visible and penalised. Your **Test Report** (D3) must show programs that actually run under D2. Your **AI Usage Journal** (D4) must be dated across the project period, not backfilled. And the **Oral Defense** will surface any mismatch between what you wrote and what you understand. The instructor strongly recommends starting early and working steadily rather than compressing everything into the final week.

4. Deliverable Specifications

D1 — Design Specification Document

A PDF report, 8–12 pages, containing the following sections in this order:

4.1 Language Overview (½ page)

Name of your language, intended domain, sample program, and a one-paragraph justification citing Sebesta's **language evaluation criteria** (readability, writability, reliability, cost — Ch. 1). Which criteria does your DSL prioritize, and which does it knowingly sacrifice?

4.2 Lexical Structure (½ page)

List of **token categories**: identifiers, literals, keywords, operators, separators. Give the regular expression or pattern for each.

4.3 Syntax (2 pages)

Complete grammar in **EBNF** (Sebesta §3.1–3.2). Must be unambiguous for the subset you implement. Annotate any places where you resolve ambiguity (e.g., operator precedence, dangling else).

```
/* Example EBNF fragment, Sebesta style */
<expression> ::= <term> { ("+" | "-") <term> }
<term>       ::= <factor> { ("*" | "/") <factor> }
<factor>     ::= <ident> | <number> | "(" <expression> ")"
```

4.4 Semantics (2 pages)

Give either **operational** or **denotational** semantics (Sebesta §3.5) for *at least two* non-trivial constructs from your language — e.g., the while loop or your domain-specific construct. Choose one style and apply it consistently. Axiomatic semantics is *not* accepted for this project.

4.5 Type System (1–2 pages)

Answer all of the following in prose, each answer tied to Sebesta Ch. 6:

- Which **primitive types** does your language have, and what are their value sets?
- Is your language **strongly typed** (§6.12)? Justify.
- Does your language allow **implicit type coercion** (§7.4)? If yes, state the coercion rules.
- What is your rule for **type equivalence** — name equivalence or structural equivalence (§6.14)? Why did you choose it?
- For your structured type (array/record/...), give the design decisions from Sebesta §6.5–6.7 that apply (e.g., for arrays: static vs. dynamic, index range, subscript check).

4.6 Names, Binding, Scope, Lifetime (1 page)

Answer, citing Sebesta Ch. 5:

- What are the rules for **legal identifiers**?
- What **bindings** happen at compile time vs. run time?
- Does your language use **static scoping** or **dynamic scoping**? Why? What would break if you switched?
- What is the **lifetime** of local variables — stack-dynamic, static, or explicit-heap-dynamic?

4.7 Expressions & Assignment (½ page)

State your decisions from Sebesta Ch. 7:

- **Operator precedence** and **associativity** table.
- Do you evaluate Boolean operators with **short-circuit** semantics?
- Order of operand evaluation — defined or unspecified?
- Assignment: is it a **statement** or an **expression**?

4.8 Design Rationale (1 page)

For each of the five numbered design decisions from sections 4.5–4.7 above, write one paragraph justifying your choice and naming at least one **trade-off** you accepted. This section is where AI-generated boilerplate fails most visibly; write in your own voice.

D2 — Implementation

Submit source code (public Git repo link). Your implementation must:

- Include a README with build/run instructions and the command line used to execute your three example programs.
- Include a working **lexer** and **parser** that accept all programs in your D3 test suite.
- Include a **type checker** consistent with the type system you declared in D1 (strong/weak typing, coercion rules, type equivalence).
- Include an **interpreter** that executes programs respecting your declared scoping rule, precedence, associativity, and parameter-passing mode.
- Produce a printable AST for each accepted program (a --dump-ast flag or equivalent).
- Reject malformed programs with a **useful error message** including line number.
- Produce an informative runtime error for: division by zero, index out of range (if applicable), and at least one domain-specific error.
- Be implementable in any of: Python, JavaScript/TypeScript, C, C++, Java, C#, Go, Rust, OCaml, Haskell. Parser generators (ANTLR, yacc/bison, pegjs) are allowed as long as you can explain them in the oral defense.

> **Consistency between D1 and D2 is graded strictly**
> Your implementation must match the language you described in D1. If your D1 says “static scoping” but your interpreter resolves variables dynamically, **both** D1 and D2 lose points, because it means you did not understand your own design. You are allowed to revise D1 — as long as the final submitted spec matches your final submitted implementation.

D3 — Example Programs & Test Report

A short PDF (3–5 pages) containing:

- **Three non-trivial example programs** in your language that together exercise every control structure, your structured type, function definition and call, and your domain-specific construct.
- For each program: the source listing, the **expected output**, and a brief (3–5 sentence) discussion of what it demonstrates about your language.
- **Five malformed programs** that demonstrate parser error recovery. For each: the source, the error message your parser produces, and the line number.
- **At least one program that exercises your type checker** — i.e., a type error that is caught before execution. Show the type error message.

D4 — AI Usage Journal

See §6 for full specification, format requirements, and the entry template. The journal is a separate PDF, 6–15 pages, with timestamped entries across the lifespan of the project. Backfilled or undated journals receive zero.

D5 — Retrospective & Self-Assessment

A single-page PDF with two sections:

- **Retrospective (½ page):** What would you change if you were to start over? What did not work and had to be scoped out of your implementation? What was the single hardest design decision, and why?
- **Self-Assessment (½ page):** Using the rubric in §5, grade yourself on each of the six dimensions (A / B / C / D). Give a one-sentence justification for each grade. A thoughtful, honest self-assessment — including identifying your own weaknesses — receives full credit; a reflexive "A across the board" does not.

5. Grading Rubric

The five written artifacts (D1–D5) are graded on the first five dimensions below. The oral defense is graded on the sixth dimension separately and carries substantial weight independent of the written artifacts.

Dimension	Excellent (A)	Good (B)	Adequate (C)	Weak (considered not submitted)
Correctness	All stated behaviors work; no contradictions between spec and implementation.	Minor bugs; ≤2 contradictions that are clearly acknowledged.	Core features work; edge cases fail; some inconsistencies with spec.	Spec and implementation diverge significantly; major features missing.
Design Justification	Every major decision justified with Sebesta terminology and a named trade-off.	Most decisions justified; terminology mostly correct.	Generic justifications; terminology used loosely.	Decisions stated without justification, or justifications are boilerplate.
Use of Sebesta Framework	Accurate use of grammar formalism, semantics style, type-system and binding terminology.	Accurate terminology with 1–2 misapplications.	Terminology mostly present but shallowly applied.	Terminology misused or absent.

Dimension	Excellent (A)	Good (B)	Adequate (C)	Weak (considered not submitted)
Originality	Clearly distinct DSL with a genuine domain-specific construct.	Distinct DSL; domain-specific construct is present but thin.	DSL is a lightly themed reskin of a generic language.	Language is indistinguishable from a generic textbook toy language.
Code Quality	Clean architecture; lexer, parser, type checker, and evaluator clearly separated.	Readable code with a working separation of concerns.	Code works but is tangled; separation of concerns unclear.	Code is opaque, undocumented, or does not match the spec.
Oral Defense	Confidently explains every choice, including alternatives not taken.	Explains most choices; recovers from follow-ups with light prompting.	Explains surface of choices; cannot discuss alternatives.	Cannot explain fundamental aspects of own submission.

How the oral defense interacts with the rest of your grade

If your oral defense demonstrates that you do **not** understand a section of your own submission at a basic level — for instance, you cannot explain what *static scoping* means in your own interpreter — the instructor reserves the right to reduce the corresponding artifact's grade accordingly. Conversely, a strong oral defense can recover points on written work that had unclear justification. ***You will be asked to write some code on the fly in your DSL. You may also be asked to modify some part of your program.***

6. The AI Usage Journal

You are required to use AI assistants on this project. It is also a graded deliverable that you document that use. The rationale is professional: modern software engineers work with AI tools as a matter of course, and a course on programming languages is a natural place to practice using them with judgment rather than blind acceptance.

Format

A single PDF, minimum 6 pages, maximum 15. Submit one entry per **session** with an AI tool. You do not need to log every trivial autocomplete — aim for the 10–20 most substantive interactions across the project. **Every entry must carry a real date**, and those dates must be spread across the working period of the project. A journal whose entries are all dated in the final week is not credible and will receive zero.

Required contents of each entry

- **Date** and which AI tool/model you used.
- **What you were trying to do** (one sentence).
- **The prompt you sent**, verbatim.
- **The relevant portion of the response** (verbatim excerpts or a tight summary).
- **What you accepted, what you rejected, and why** — this is the graded part.
- **Any errors or misleading claims the AI made**, and how you detected them.
- **Most importantly,** include the web link to your conversation with the AI model**.**

Three mandatory experiments

At least three of your journal entries must document the following experiments:

#	Experiment	What to report
E1	Ask the AI to generate an EBNF grammar for your language from a one-paragraph description.	Compare its grammar with yours. What did it get wrong or suboptimal? Does it handle operator precedence correctly?
E2	Ask the AI to explain the difference between **name equivalence** and **structural equivalence** for your record type.	Does its explanation match Sebesta §6.14? Does the example it gives actually run under the equivalence rule it described?
E3	Ask the AI to implement your type checker (or a non-trivial portion of it).	Run its code on at least three test inputs where you know the right answer. Report any bugs, subtle or blatant, and how you fixed them.

How the journal is graded

- **Honesty.** Fabricated or copy-pasted-for-show entries are worse than none and will be treated as academic dishonesty.
- **Depth of reflection.** Entries that merely say “AI gave correct answer, I used it” get minimum credit. Entries that discuss why the answer was correct, what could have gone wrong, or how you verified it, get full credit.
- **Critical evaluation.** The journal should contain at least three documented cases where the AI was wrong, unclear, or suboptimal.

> **What is not allowed**
>
> You may not paste AI-generated text directly into your Design Specification, Final Report, or AI Usage Journal reflections without marking it as a quotation and discussing it. The *rationale* sections and the *reflection* parts of the journal must be in your own voice. AI-assisted **code** is fine — but you must be able to explain every line you submit.

7. Sample Language Specification: “ShapeScript”

What follows is a **deliberately short example** of the kind of spec we expect — *shorter* than yours will be, so you can see the shape of it without being tempted to copy its design choices. Your spec must make **different** choices from this one in at least three places. (Otherwise your domain probably isn't different enough.)

7.1 Overview

ShapeScript is a DSL for procedurally describing 2D geometric compositions. It prioritizes **writability** (Sebesta §1.3.2) for domain users and **reliability** via strong typing over raw numeric performance, which it knowingly sacrifices.

```
shape flower(petals: int) {
  repeat petals times {
    draw circle(radius: 10)
    rotate 360 / petals degrees
  }
}
draw flower(petals: 8)
```

7.2 Lexical Structure (excerpt)

```
IDENT     = [a-zA-Z_][a-zA-Z0-9_]*
INT_LIT   = [0-9]+
FLOAT_LIT = [0-9]+ "." [0-9]+
KEYWORDS  = shape | draw | repeat | times | rotate | degrees | if | else
OPERATORS = + - * / = == != < <= > >=
SEPARATORS= ( ) { } , :
```

7.3 Syntax (excerpt, EBNF)

```
<program>   ::= { <shape_decl> | <stmt> }
<shape_decl>::= "shape" IDENT "(" [ <params> ] ")" <block>
<params>    ::= IDENT ":" <type> { "," IDENT ":" <type> }
<stmt>      ::= <draw_stmt> | <repeat_stmt> | <rotate_stmt> | <if_stmt>
<repeat_stmt>::= "repeat" <expr> "times" <block>
<expr>      ::= <term> { ("+" | "-") <term> }
<term>      ::= <factor> { ("*" | "/") <factor> }
```

7.4 Type System

- **Primitive types:** int, float, bool.
- **Structured type:** point, a record with fields x: float and y: float.
- **Strong typing:** yes. Every operator is defined only on matching operand types.
- **Coercion:** int is implicitly widened to float in mixed-mode arithmetic; float is **not** narrowed to int.
- **Type equivalence:** name equivalence for records — two record types are equivalent only if they have the same declared name.

7.5 Scope, Binding, Lifetime

- **Scoping:** static (lexical). Each shape body opens a new scope.
- **Bindings:** variable types are bound at declaration time. Shape definitions are bound before the first draw.
- **Lifetime:** parameters and locals are stack-dynamic; shape declarations have static lifetime (whole program).

7.6 Expressions

- **Precedence:** unary minus > * / > + − > comparison > && > ||.
- **Associativity:** all binary operators are left-associative.
- **Short-circuit:** && and || short-circuit left-to-right.
- **Operand evaluation order:** left-to-right and fully defined, for reliability.
- **Assignment:** a statement, not an expression — no assignment-in-expression surprises.

7.7 Semantics (operational, sketch)

Let *S* be the state (current turtle position, heading, canvas). The semantics of repeat n times B is, in big-step operational form:

```
    <n, S> -> n'       <B, S>  -> S1
    <B, S1> -> S2  ...  <B, S_{n'-1}> -> S_{n'}
    ----------------------------------------
         <repeat n times B, S> -> S_{n'}
```

Note how this reveals a design decision: n is evaluated once, not each iteration. Changing this would change the semantics; the design rationale must explain the choice.

7.8 Design Rationale (excerpt)

Why name equivalence for records? Structural equivalence would conveniently let users pass any {x, y}-shaped record to a function expecting point, but it would also silently accept any accidental {x, y}-shaped record that was intended to represent a *vector* or a *dimension*. For a DSL aimed at correctness in a geometric setting, name equivalence catches this confusion at type-check time. The trade-off accepted: more point constructor calls in user code.

8. AI Usage Journal — Entry Template

Copy this template for every journal entry. Entries should be concise (½ to 1 page each). You must have entries from at least **three distinct weeks** of working on the project.

Field	Content
Entry #	e.g., 7 of 14
Date	YYYY-MM-DD (must be a real date during project work)
Phase	Design / Lexer / Parser / Type checker / Interpreter / Testing / Writing
AI tool	Claude Opus 4.7 / ChatGPT / Copilot / Gemini / other (+ version if known)
Goal	One sentence — what you were trying to achieve. *Example: "Write the pretty-printer for my AST."*
Prompt	Verbatim. If you iterated, show the final prompt plus a one-line summary of what changed.
Response (key part)	Verbatim code/text, or a tight summary with direct quotes of the important lines.
Accepted	What you kept, and **why**.
Rejected / modified	What you threw away or changed, and **why**. Be specific: bug, style issue, wrong abstraction, hallucinated API, etc.
Errors you caught	Any mistakes the AI made. How did you detect them? What would have happened if you hadn't?
Reflection (2–3 sentences)	What did you learn about the problem — or about how to use AI — from this session?
AI Conversation Link	All AI models have the share feature. Do not delete your sessions with your AI model, get the conversation link and include it for each reported session.

Worked example entry (for reference)

Sample entry

Entry # 4 of 12 · **Date** 2026-04-02 · **Phase** Parser · **AI tool** Claude Opus 4.7

Goal. Write the recursive-descent parser for the expression grammar.

Prompt. "Here is my EBNF grammar for expressions [pasted]. Write a recursive-descent parser in Python that returns an AST with classes BinOp, UnaryOp, NumLit, Ident. No parser combinator libraries."

Accepted. The overall structure (one function per non-terminal) and the BinOp(op, left, right) AST shape — matches my design.

Rejected / modified. The AI made parseExpr right-associative by recursing on itself before consuming the operator. My grammar specifies **left**-associativity. I rewrote it as a loop that consumes operators in sequence.

Errors caught. (1) Right-associativity bug above. (2) It omitted handling of unary minus entirely. Caught by running on the test input -3 + 2, which parsed as - (3 + 2).

Reflection. AI is good at boilerplate parser structure but unreliable on precedence/associativity decisions — exactly the parts that encode the language designer's intent. Next time I'll include a precedence table in the prompt and check associativity with a dedicated test before accepting anything.

Conversation link: https://claude.ai/share/7aa815dd-dea2-4ec8-9242-4326b7a1e098

9. Academic Integrity

- AI assistance is **required and encouraged**, but must be **honestly documented**.
- Copying another student's design, code, or rationale is plagiarism.
- Using AI to generate a design rationale that you cannot defend in the oral viva is treated as a failure of the viva and will propagate to the written grade.
- Two submissions whose designs are suspiciously similar will both be investigated. Make original choices — that is the whole point of the project.
- If in doubt, ask the instructor before submitting.

Final note

This project is designed to be **genuinely hard to game** and **genuinely educational**. If you treat it as an AI-copy-paste exercise, you will struggle in the oral defense. If you treat it as what it is — a chance to make real programming-language design decisions and defend them — you will come out of it understanding every chapter of the book in a way no exam can teach. Good luck.